

Simulating The Mount Rainier Scenic Railroad Object Detection & Response

*Department of Mechanical and Aerospace Engineering
Princeton University, Princeton, NJ 08544*

Sonal Bhatia, Hriday Unadkat, & Sanjana Venkatesh

May 11, 2026

Abstract

This project was inspired by the Mount Rainier Scenic Railroad, which has suffered several track closures recently due to fallen trees. We build a model train setup in which sensors detect fallen trees, and the system controls trains' motion accordingly. The system integrates a custom-built 6502-based vector-board computer, an Arduino co-processor, VL53L0X time-of-flight (ToF) sensors, servo-actuated obstacles, relays, and motorized turnout switches into a closed-loop control architecture built on a physical model railroad layout using Atlas flex track. Ultimately, the project aimed to demonstrate a mix of sensing, actuation, and control principles in furtherance of the core goal of a train maintaining safety in a rigid constrained path on a predefined test bench.

This report represents our own work in accordance with University regulations. /s/ SB, HU, SV

1 Introduction

The Mount Rainier Scenic Railroad is a heritage steam railroad operating on a 7-mile (14 mile roundtrip) section of the historic Tacoma Eastern Railway line near Mount Rainier, Washington. Running through dense forest and across four looped lines, the railway offers scenic views of the Nisqually River and Mount Rainier to its passengers. In 2023, the railroad reopened after a considerable hiatus caused, in part, by unusually heavy rainfall that downed numerous trees across the track. Staff and volunteers were required to manually remove the fallen debris and inspect the track before service could resume [6]. Hazards of this sort, ones where obstructions appearing on the track with little warning, pose a significant and persistent risk to both operational and passenger safety on heritage lines operating through the terrain.

Motivated by this scenario, this project develops a scaled, autonomous model train system that integrates **sensing, actuation, and closed-loop control** to detect and respond to fallen-tree hazards in real time. The system is built on a simplified track layout inspired by the loops of the Mount Rainier Scenic Railroad. The track is designed to identify obstructions using time-of-flight (ToF) distance sensing, coordinate a control response that halts or reroutes trains around the obstacle, and intelligently sequence train operations to preserve the safety of simulated passengers onboard. The entire train-track-computer system operates autonomously without human intervention during a hazard event.

The primary objectives of this project are as follows:

1. Reliable *sensing* of track obstructions using ToF sensors with sufficient accuracy and response time to prevent collisions.
2. Repeatable *actuation* of a servo-drive pulley mechanism capable of introducing and removing obstacles from the track.
3. *Closed-loop control* that halts or reroutes the train upon obstruction detection.

Future work includes extensions audio signaling for fully blocked track-conditions and vision-based object recognition using an onboard camera.

Together, these components will form an end-to-end demonstration of how embedding sensing and control can be applied to a real-world rail safety problem.

The following sections of this report describe the engineering design, concept of operations, system architecture, hardware and software design, control logic, and evaluation of the implemented prototype.

2 Engineering Design

The Accreditation Board for Engineering and Technology, Inc. (ABET) definition of design is as follows:

”Engineering design is the creative process of devising a system, component, or process to meet desired needs. It is a decision-making process (often iterative), in which basic scientific knowledge and mathematics are applied to meet, optimally, a stated objective. Core elements of design are a clear statement of objectives and criteria, synthesis, analysis, realization (construction), testing and evaluation.”

The project endeavors to design an autonomous railroad safety system capable of detecting track obstructions, actuating their removal, and coordinating train response in real time. In accordance with the ABET definition, the design process followed here is iterative: requirements were established from the motivating Mount Rainier scenario, constraints were identified from the available hardware and course environment, and design drivers were used to guide trade-off decisions across sensing, actuation, and control subsystems. The following subsections state these requirements, constraints, and drivers explicitly, along with a commentary on the iterative design process, thus providing the basis against which the synthesis, realization, and testing described later in this report are evaluated.

2.1 Requirements

The system shall meet the following functional requirements, derived from the primary objectives of the project:

1. **Obstruction Detection.** The system shall detect a track obstruction within a defined hazard zone using time-of-flight (ToF) distance sensing, with response latency short enough to halt the train before collision at nominal operating speed.

2. **Obstacle Actuation.** The system shall introduce and remove a simulated tree obstacle from the track in a controlled, repeatable manner via a servo-driven pulley mechanism, with vertical clearance orthogonal to the project board, thus fully clear for the loading gauge of the train.
3. **Closed-Loop Control.** Upon detection of an obstruction, the system shall autonomously halt the train or reroute it via a turnout to avoid the hazard, without human intervention during the event.
4. **Recovery.** Following obstacle removal, the system shall verify track clearance via the ToF sensor and resume nominal train operation.
5. **Fail-Safe Behavior.** If all available paths are obstructed, the system shall halt all trains and remain in a safe state until the hazard is cleared.

2.2 Constraints

The design is subject to the following constraints:

- **Sensing Constraint.** The time-of-flight (ToF) sensing creates varied distance readings, motivating a constraint regarding the minimum distance required to initiate an alternate control action for the train.
- **Spatial Constraint (Hazard Zone Definition).** Obstruction detection must occur within a defined hazard zone ahead of the train, constraining sensor placement, field-of-view, and stopping distance requirements.
- **Closed-Loop Timing Constraint.** The system must execute detection-to-action decisions in real time without human intervention.
- **Path Constraint.** Track routing logic is constrained by available turnouts, which are themselves constrained by the maximum angle the Atlas track permits (flextrack) and the size of the wood board to which the system is mounted.
- **Recovery Validation Constraint.** After obstacle removal, the system must verify track clearance using the ToF sensor before resuming operation, ensuring safe restart is sensor-validated rather than time-based.
- **Reliability and Repeatability Constraint.** Obstacle insertion and removal must be mechanically repeatable across cycles.

2.3 Drivers

The design is subject to the following drivers:

- **Detection reliability over sensing range.** Because hazard zones are localized and well-defined in the scaled layout, a narrow-beam, low-noise sensing modality (ToF) was preferred over broader-beam ultrasonic sensing, even at the cost of a smaller field of view. This drove the selection of the VL53L0X over alternative range-finding technologies.
- **Closed-loop response time.** The full sense-decide-actuate loop must complete within the time it takes the train to traverse the hazard zone at nominal speed, driving the choice of sensor sampling period (500 ms), the use of continuous-mode I²C reads, and the partitioning of safety-critical logic onto a dedicated Arduino loop separate from the supervisory 6502.
- **Repeatability.** The obstacle deployment mechanism must place the tree in a consistent position across trials so that the ToF threshold can be tuned reliably, driving the choice of a servo-driven pulley with fixed mounting geometry and 3D-printed mounts dimensioned to the specific servo body.

2.4 Iterative Design Process

The design process for this project is explicitly iterative, consistent with the ABET framing of engineering design as a decision-making process in which objectives and realizations are repeatedly re-evaluated against one another. Rather than proceeding linearly from requirements to a single realized design, the project advances through cycles of specification, analysis, bench-testing, and revision, with each cycle informing the next.

Several iterations have already shaped the design presented in this report:

- **Sensor modality selection.** The sensing subsystem was initially scoped to use ultrasonic range finding, which is inexpensive and familiar from prior coursework. Preliminary analysis of beam profile and environmental noise susceptibility led to revisiting this choice, and the design was revised to use time-of-flight (ToF) sensing for its narrower beam and improved reliability in the cluttered track environment.
- **Track layout reduction.** The initial concept mirrored the Mount Rainier Loop more literally, with a main loop and three peripheral loops. Iterative schematic design in AnyRail revealed that the full four-loop layout could not satisfy the minimum curve-radius and angular constraints of the Hornby track segments within the project board footprint. The layout was accordingly reduced to a main loop plus a single secondary loop, preserving the functional character of the original (multiple routes, a reroutable hazard zone) within the available space.
- **Unified obstacle deployment and removal.** Early versions of the design treated obstacle introduction and obstacle removal as separate mechanisms. This was revised, on reflection, to a single servo-driven pulley subsystem that handles both functions, reducing part count, simplifying control logic, and improving positional repeatability between trials.

3 Concept of Operations

The system operates as a self-contained demonstration of autonomous hazard response on a scaled heritage railroad. Figure 1 shows the track layout, which consists of three loops: a default (main) loop, an inner loop, and an outer loop, interconnected by four motorized turnouts. Two designated hazard zones are instrumented with VL53L0X time-of-flight sensors, with one on the inner loop (Track 4) and one on the outer loop (Track 5). A 3D-printed tree obstacle (Figure 3) is mounted near each hazard zone and is periodically lowered by a servo onto the track at randomized intervals to simulate fallen and subsequently cleared trees.

The route takes the train through the outer loop (Figure 1, orange segment), then the inner loop (Figure 1, turquoise segment), then off of the center board via Figure 1, green segment. When the inner-loop tree falls and the ToF sensor on the inner loop registers an obstruction (distance ≤ 120 mm), the Arduino immediately cuts power to the inner-loop track segment via relay R_{inn} . When the outer-loop tree falls and the sensor on Track 5 registers an obstruction (distance ≤ 170 mm), power to the outer-loop segment is cut via relay R_{out} . Once the tree is raised and the sensor reading returns above its threshold, the affected relay is re-powered and the train resumes operation.

Table 1 summarizes the possible turnout switch configurations for each of the four switches, their Arduino pin assignments, and their directional behavior under each routing pattern.

Table 1: Turnout Switch Configuration and Behavior

Switch	DIR Pin	TRIG Pin	DIR State	Behavior
SW1	4	6	LOW	Default straight (no reroute)
SW1	4	6	HIGH	Exit loop $\rightarrow$ default straight
SW2	20	7	LOW	Default $\rightarrow$ default
SW2	20	7	HIGH	Default $\rightarrow$ inner loop
SW3	21	12	LOW	Outer $\rightarrow$ default
SW3	21	12	HIGH	Inner $\rightarrow$ default
SW4	5	A0	LOW	Default $\rightarrow$ default (exit board)
SW4	5	A0	HIGH	Default $\rightarrow$ outer loop

The three routing patterns are enacted by setting all four DIR lines simultaneously and then triggering all four switches in a staggered sequence (5 ms between each trigger pulse) to avoid current spikes from simultaneous solenoid firing. The patterns are defined as follows and cycle sequentially on fixed timers:

- **Pattern 1** (60s): [LOW, LOW, LOW, HIGH] - train routed onto the outer loop.
- **Pattern 2** (30s): [HIGH, HIGH, HIGH, HIGH] - train routed onto the inner loop.
- **Pattern 3** (30s): [HIGH, LOW, HIGH, LOW] - train returned to the default loop.

Track safety is evaluated on every control loop iteration independent of the pattern sequencer, so a hazard event can cut power to the affected segment at any point during the demonstration cycle without waiting for a pattern transition. The remainder of this report describes how each element of this operational picture is realized in hardware and firmware.

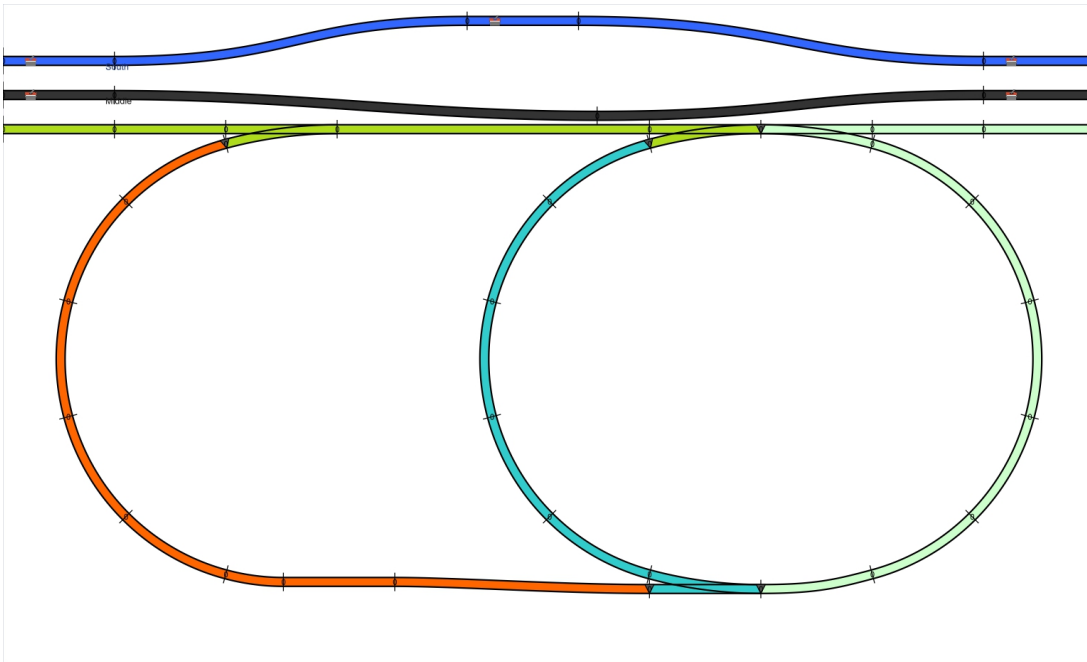


Figure 1: Track Layout in AnyRail

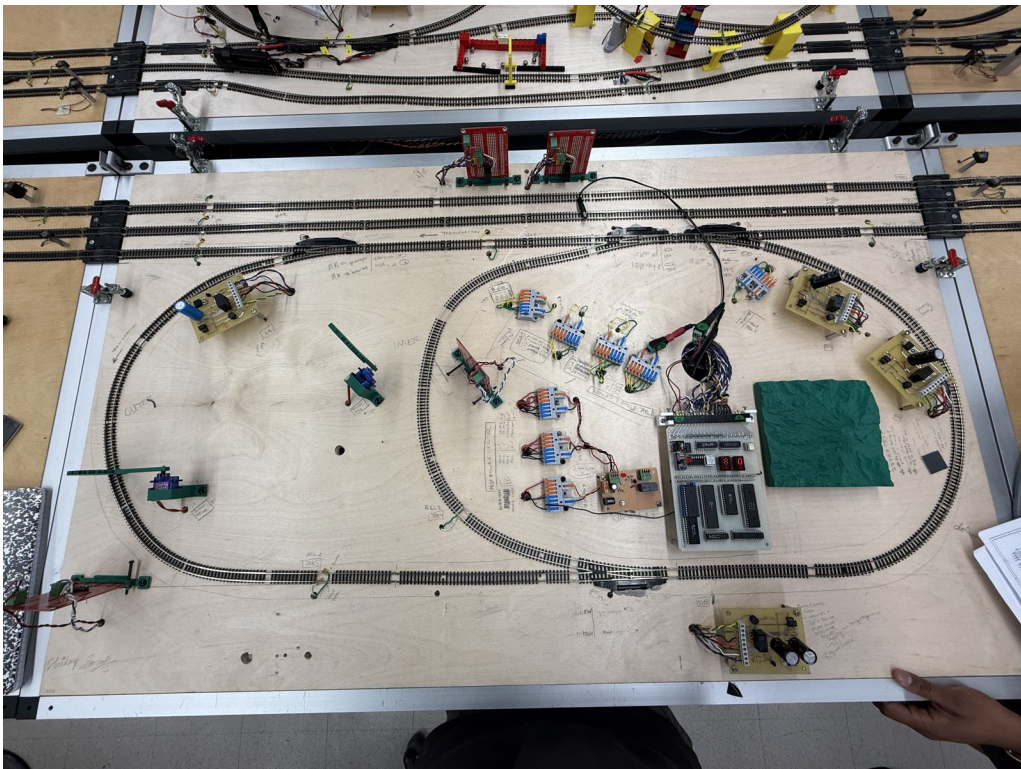


Figure 2: Physical Track Layout & Project Design

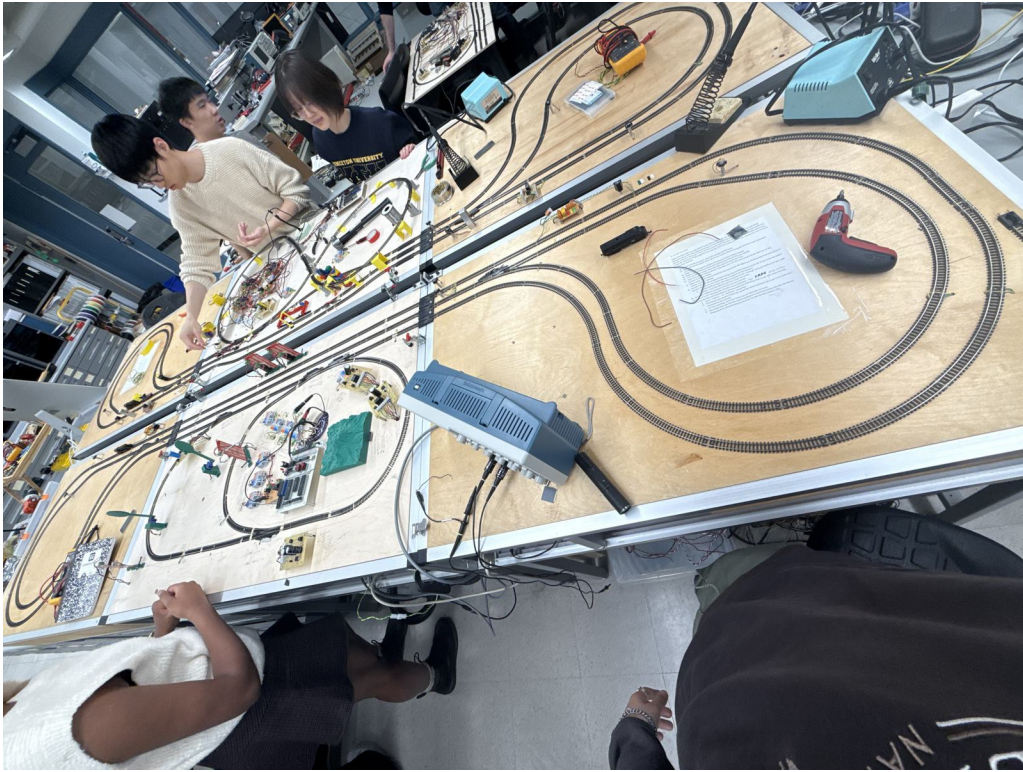


Figure 4: Project Board on Test Stand



Figure 3: Obstacle (3D Printed Tree)

4 Methodology

The following subsections are designed to be reproducible. For each subsystem, we document the specific components used, circuit topology and wiring, the mechanical geometry and fabrication parameters, and the firmware entry points

that tie the hardware to the control logic.

All custom artifacts of the project (CAD files, firmware source, GAL maps, etc.) are maintained in the project repository linked here and referenced throughout.

4.1 Computer Design

The centerpiece of the control system is a custom-built 8-bit microcomputer, assembled on a vector board and functionally equivalent to the 1976 Apple 1. The architecture follows the 6502-based microcomputer pattern: a single MPU communicates with memory and I/O devices over a shared 16-bit address bus and 8-bit data bus, with address decoding performed by a Generic Array Logic (GAL) chip to generate chip-select signals for each memory mapped device. A system clock drives the MPU and memory, while an independent higher-frequency oscillator drives the serial interface. An Arduino [1] serves as a co-processor that handles the real-time sensing and actuation tasks (time-of-flight reads, servo PWM, turnout switching, and relay control), with the vector-board computer providing higher-level supervisory and display functions. Two TIL displays provide on-board visual output. The complete assembly of the vector board is presented in Figure 5.

4.1.1 Components

Electrically-Erasable Programmable Read-Only Memory (EEPROM) A 28C64B 8 kB EEPROM [3] holds the non-volatile program code (the “firmware”) that executes on power-up. The 6502 reset vector is stored at the top of EEPROM address space so that on reset the MPU fetches the starting address of the main program. Firmware is authored as 6502 assembly source (.asm) files and assembled into raw binary (.bin) images using the vasm cross-assembler [12] with its 6502 syntax module. The resulting binary is then written to the EEPROM off-board using a SuperPro universal device programmer. Once programmed, the EEPROM is re-seated into its socket on the vector board, and the firmware executes on the next power-up or reset.

Random-Access Memory (RAM) A 6116 static RAM (SRAM) chip [4] provides 2 kB of volatile working memory for the 6502, used for the processor stack, the zero page, and general-purpose scratch storage during program execution. The RAM is memory-mapped into the lower portion of the address space, with the GAL asserting the 6116’s chip-select line whenever the MPU places an address in that range on the bus.

Microprocessor Unit (MPU) The MPU is a Synertek SY6502 [10], a second-source NMOS implementation of the MOS 6502 used in the Apple 1 and functionally identical to it in instruction set and pinout. The SY6502 is an 8-bit processor with a 16-bit address bus supporting up to 64 kB of memory-mapped address space. All computation, bus control, and instruction sequencing are performed by this chip; every other component on the board is a peripheral that responds to the addresses and read/write signals that the MPU places on the shared buses.

Versatile Interface Adapter (VIA) A 6522 VIA [7] provides two 8-bit bidirectional parallel I/O ports (PA and PB), each with associated handshake lines, as well as two programmable 16-bit timers and a shift register. The VIA is the primary general-purpose I/O device on the board and is used in this project to drive the relay and servo control lines, to read the push-button input that triggers obstacle deployment, and to source digital signals to the TIL display.

Asynchronous Communications Interface Adapter (ACIA) A 6551 ACIA [8] provides an RS-232-compatible asynchronous serial port. It is used to communicate between the vector-board computer and a host PC (for program download and debug output) and, more importantly for this project, to participate in the course-wide serial data network through which microcomputers on adjacent track segments coordinate with one another.

Two Clocks: 1 MHz & 1.8432 MHz The 1 MHz clock acts as the main system timing source that synchronizes the CPU, memory, and other digital components. Each clock pulse advances the processor by one step, controlling when instructions are fetched, decoded, and executed. The 1.8432 MHz clock, connected to the ACIA, is used for serial communication because it can be divided evenly into standard baud rates such as 9600, 19200, and 115200 bits per second. This precise timing ensures accurate transmission and reception of serial data between the computer and external devices like terminals or modems. While the 1 MHz clock governs general computer processing, the 1.8432 MHz clock provides the stable timing needed for reliable asynchronous communication.

Gate-Array Logic (GAL) A GAL16V8 programmable logic device [5] performs address decoding for the board. The GAL takes selected high-order address lines and the MPU's clock and read/write signals as inputs and produces active-low chip-select outputs for the EEPROM, 6116 RAM, 6522 VIA, and 6551 ACIA, ensuring that exactly one device drives the data bus at any given time. Using a GAL instead of a fixed network of discrete logic gates allows the memory map to be changed by reprogramming a single chip, which proved important during iterative debugging.

Arduino A single Arduino ATmega328P microprocessor [2] serves as a dedicated I/O co-processor for the sensing and actuation subsystems. The Arduino interfaces directly with the two VL53L0X time-of-flight distance sensors (one per hazard zone) over a shared I²C bus, drives the PWM control signals for the two servo motors that raise and lower the tree obstacles, and gates track power through two relay outputs. It also issues turnout-switching commands on a set of dedicated digital pins that drive the four motorized turnout solenoids. The Arduino executes the full sensing-to-actuation control loop autonomously and in real time; the SY6502 vector board computer participates in higher-level supervisory tasks such as displaying operating state on the TIL outputs and coordinating with adjacent track segments over the course-wide ACIA serial network, while delegating the timing-sensitive peripheral I/O to the ATmega328P.

Buffers 74LS244 octal buffer/line drivers [11] are used to isolate the MPU's address bus from external loads. Buffering is necessary because the SY6502's output drive current is limited; without buffers, the capacitive and resistive loading imposed by the relay, servo, and sensor interfaces would degrade bus signal integrity and produce unreliable reads and writes. The 74LS244 is a unidirectional, non-inverting buffer appropriate for the address bus, which is always driven by the MPU and never needs to carry data back toward it.

TIL Display Two TIL displays provide on-board visual output that is used to display train barcode information that is read from sensors on the test stand.

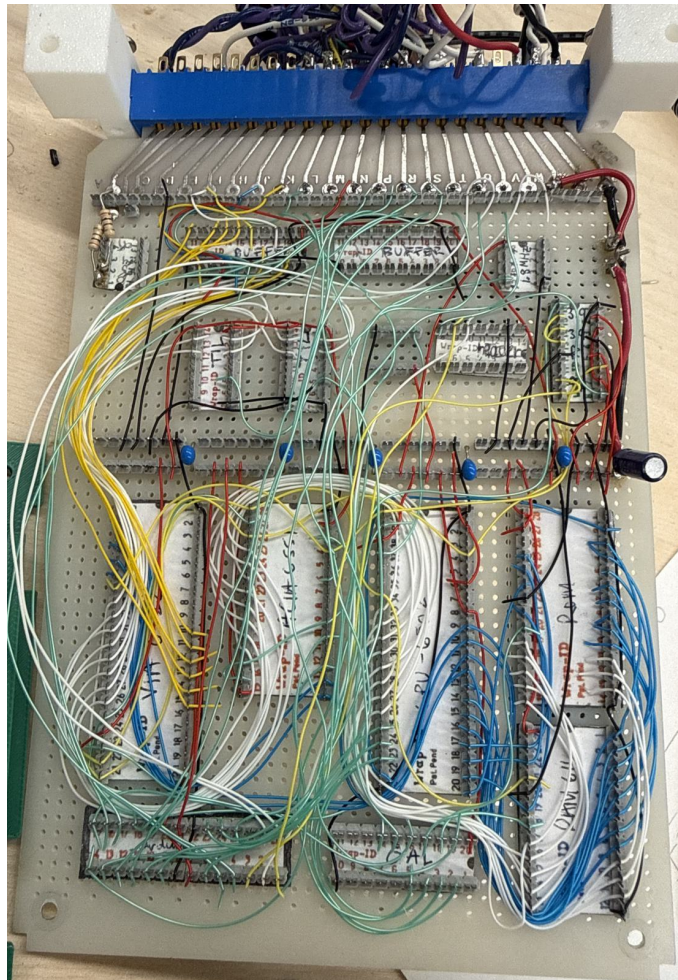


Figure 5: Vector Board Computer

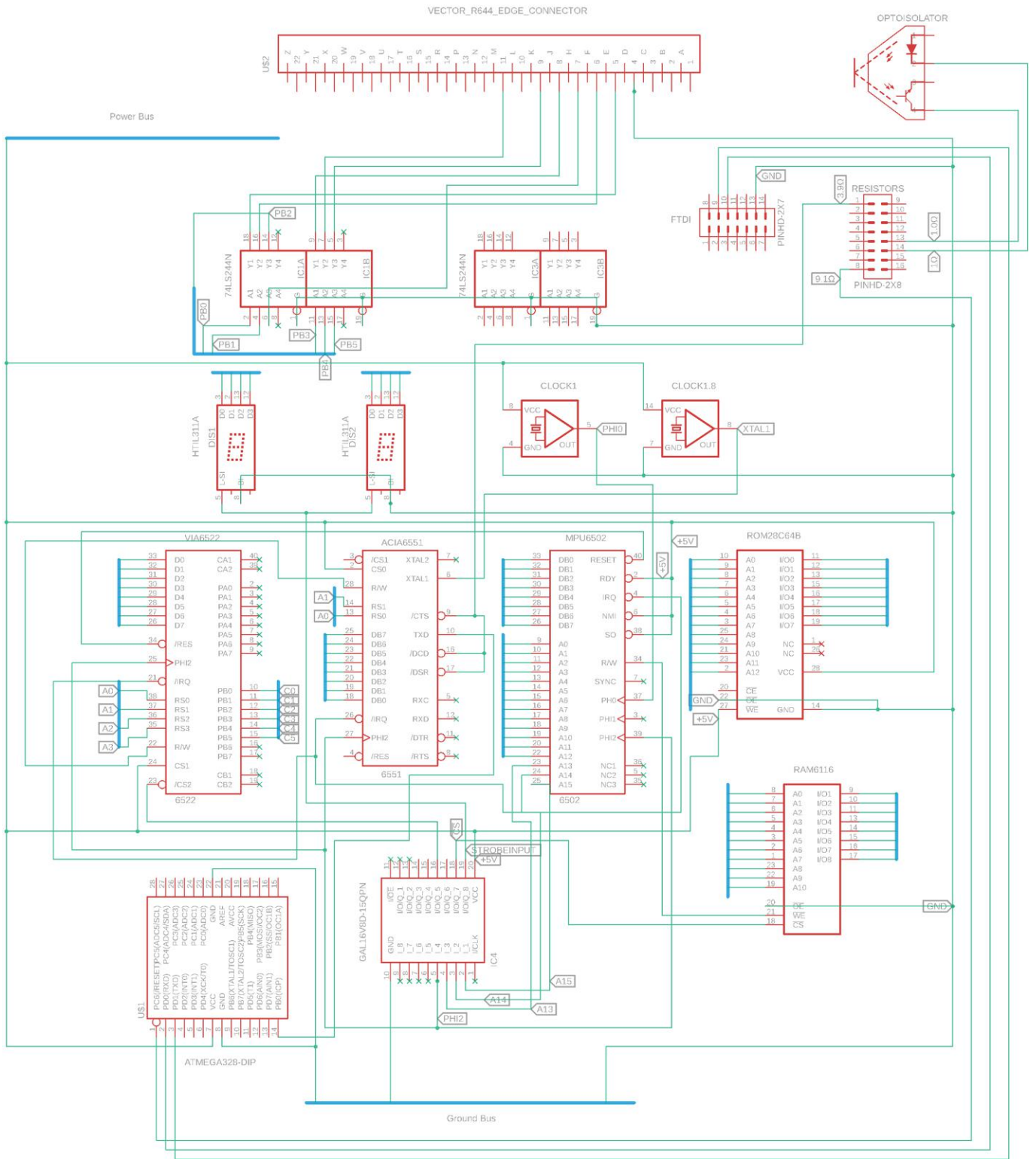


Figure 6: Electrical Schematic of Computer

4.2 Electrical Circuit Design

4.2.1 Sensing: Computer-Sensor

Control Logic Pseudocode The control logic for the dual-hazard-zone scenario is implemented entirely on the sensing/actuation Arduino, which samples two VL53L0X ToF laser-ranging sensors [9], one mounted at each hazard zone, and directly drives the track-power relays and servo motors in response. The outer sensor (S_{out} , I²C address 0x30) monitors the outer loop hazard zone, and the inner sensor (S_{inn} , address 0x31) monitors the inner loop hazard zone. Because the two sensors share the same I²C bus, their XSHUT lines are held low at startup and each device is brought up sequentially so that a unique address can be assigned before the next sensor is initialized.

The obstruction thresholds are tuned independently for each zone: a reading at or below 170 mm triggers the outer-zone flag, while a reading at or below 120 mm triggers the inner-zone flag. The asymmetry reflects differences in mounting geometry and baseline clearance between the two installations. On every iteration of the main loop (which runs at approximately 500 ms owing to the fixed `delay(500)` at the loop’s end), the firmware reads both sensors in continuous mode, evaluates the two flags, and writes the corresponding relay outputs. Algorithm 1 captures this response, and all code for this project is available in the Github repository linked here.

Algorithm 1 Dual-zone hazard detection and track-power response

Require: Outer ToF sensor S_{out} at the outer hazard zone with threshold $d_{\text{out}} = 170$ mm

Require: Inner ToF sensor S_{inn} at the inner hazard zone with threshold $d_{\text{inn}} = 120$ mm

Require: Track-power relay control lines (relays active LOW) R_{inn} , R_{out} on pins A1 and A2, controlling the inner and outer track segments

```
1: Initialize:  $R_{\text{inn}} \leftarrow \text{HIGH}$ ,  $R_{\text{out}} \leftarrow \text{HIGH}$  ▷ both tracks powered on
2: loop ▷ main control loop, runs every 500 ms
3:    $d_{\text{out}} \leftarrow \text{READRANGECONTINUOUS}(S_{\text{out}})$ 
4:    $d_{\text{inn}} \leftarrow \text{READRANGECONTINUOUS}(S_{\text{inn}})$ 
5:    $\text{outerFallen} \leftarrow (d_{\text{out}} \leq 170)$ 
6:    $\text{innerFallen} \leftarrow (d_{\text{inn}} \leq 120)$ 
7:    $R_{\text{inn}} \leftarrow \text{if innerFallen then Low else High}$ 
8:    $R_{\text{out}} \leftarrow \text{if outerFallen then Low else High}$ 
9:   if innerFallen and outerFallen then
10:      $\text{SIGNAL}(\text{BOTHTRACKSCUT})$  ▷ “BOTH TRACK 4 AND TRACK 5 HAVE BEEN CUT”
11:   else if innerFallen then
12:      $\text{SIGNAL}(\text{INNERCUT})$  ▷ “TRACK 4 CUT - INNER OBSTACLE DETECTED”
13:   else if outerFallen then
14:      $\text{SIGNAL}(\text{OUTERCUT})$  ▷ “TRACK 5 CUT - OUTER OBSTACLE DETECTED”
15:   end if
16: end loop
```

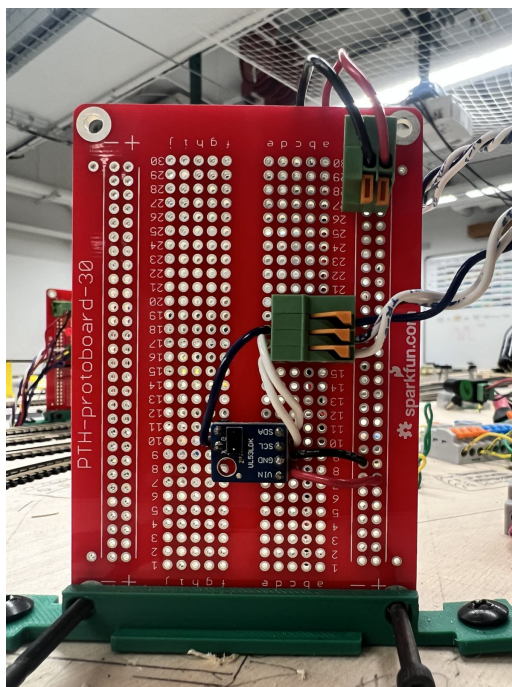


Figure 7: Time of Flight Sensor Setup

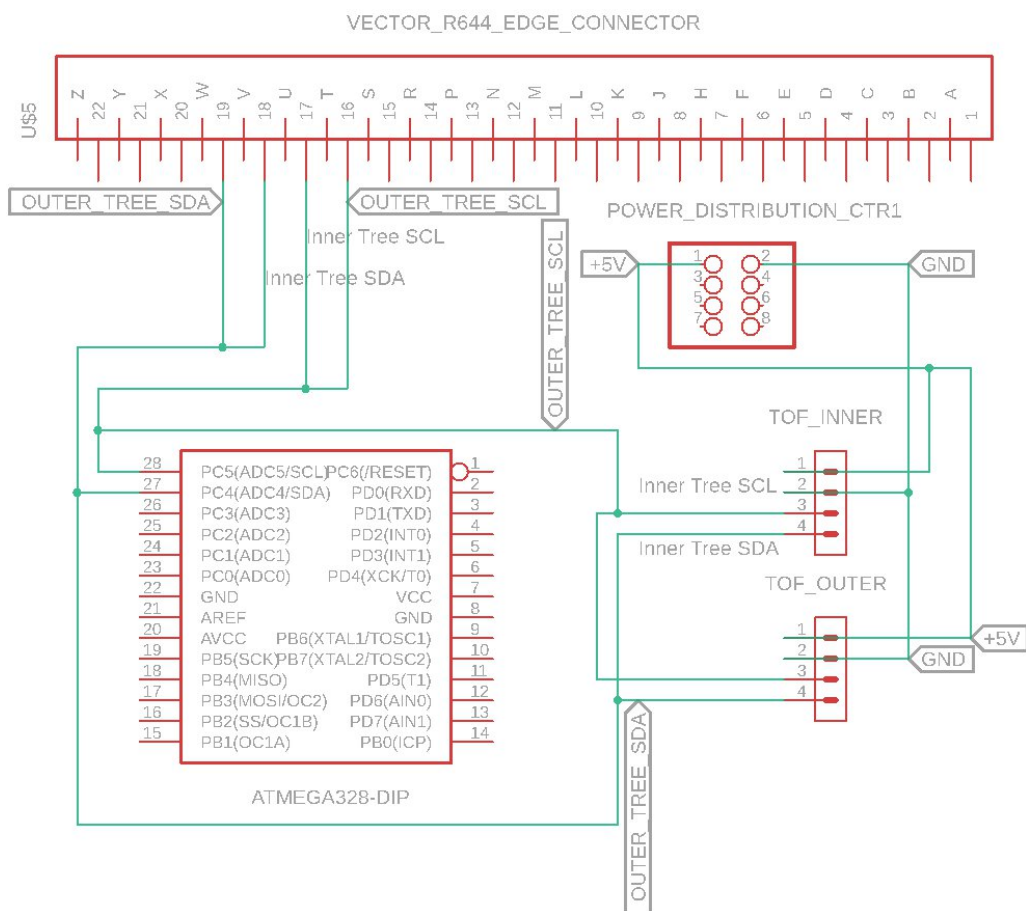


Figure 8: Time of Flight Sensor Electrical Schematic

4.2.2 Actuation: Computer-Servo

Each tree obstacle is raised and lowered by a hobby servo motor, with the servo horn screwed to a 3D-printed tree and mounted beside the track. Two servos are used, one per hazard zone: the inner servo is driven on digital pin 11 and the outer servo on pin 10, both through the standard Arduino `Servo` library. Each servo sweeps between 0° (tree raised, track clear) and 110° (tree lowered, track blocked). Motion is executed by a dedicated `moveServoSlow()` helper that steps the servo one degree at a time with a fixed `delay(15ms)` between steps, yielding a smooth sweep at approximately $67^\circ/\text{s}$.

The deployment timing is event-driven and randomized. After an initial 5 s hold phase that allows all subsystems to complete initialization, each servo is assigned an independent countdown timer. When a timer expires, the servo's target position is toggled and `moveServoSlow()` executes the full sweep to the new position; the timer is then restarted with a freshly drawn random interval. The inner servo draws its first interval from [1 s, 5 s] and all subsequent intervals from [5 s, 10 s]; the outer servo draws all intervals from [5 s, 10 s]. The differing initial ranges introduce a natural phase offset between the two servos without requiring a hard-coded delay constant, ensuring that the two hazard zones do not toggle synchronously at the start of a demonstration run. The randomized inter-event intervals further decorrelate the two zones over time, producing an obstacle sequence that more closely resembles the unpredictable timing of real fallen-tree events on the Mount Rainier line.

Because `moveServoSlow()` uses blocking `delay()` calls, the main loop is paused for the duration of each sweep (up to approximately 1.65 s for a full 110° stroke). This is an acceptable trade-off for the present application: each sweep is short, sensor reads and relay updates resume immediately afterward, and the straightforward blocking structure simplifies the code considerably relative to a full non-blocking state machine. Algorithm 2 summarizes the actuation logic.

Algorithm 2 Dual-servo obstacle actuation with slow sweep

Require: Inner servo s_{11} on pin 11, outer servo s_{10} on pin 10

Require: Sweep range $[0^\circ, 110^\circ]$; per-step delay $\tau = 15\text{ ms}$; initial hold $t_{\text{hold}} = 5\text{ s}$

Require: Random interval ranges: inner [1 s, 5 s] on first trigger, thereafter [5 s, 10 s]; outer [5 s, 10 s]

```

1: Setup:
2:   Attach  $s_{11}$  to pin 11; attach  $s_{10}$  to pin 10
3:    $s_{11}.\text{WRITE}(90)$ ;  $s_{10}.\text{WRITE}(0)$  ▷ initial positions
4:   Seed RNG; record  $t_{\text{start}} \leftarrow \text{millis}()$ 
5: loop ▷ main loop
6:   if  $\text{millis}() - t_{\text{start}} < t_{\text{hold}}$  then continue
7:   end if ▷ hold phase
8:   if not yet started then
9:      $\text{innerTimer} \leftarrow \text{millis}()$ ;  $\text{outerTimer} \leftarrow \text{millis}()$ 
10:     $\text{innerInterval} \leftarrow \text{RANDOM}(1000, 5000)$ ;  $\text{outerInterval} \leftarrow \text{RANDOM}(5000, 10000)$ 
11:  end if
12:  for each servo  $i \in \{\text{inner}, \text{outer}\}$  do
13:    if  $\text{millis}() - \text{timer}_i \geq \text{interval}_i$  then
14:       $\text{state}_i \leftarrow \neg \text{state}_i$  ▷ toggle tree position
15:       $\text{startPos} \leftarrow \text{current angle}$ ;  $\text{endPos} \leftarrow \text{state}_i ? 110 : 0$ 
16:       $\text{MOVESERVOSLOW}(s_i, \text{startPos}, \text{endPos}, \tau)$  ▷ blocking sweep at  $\sim 67^\circ/\text{s}$ 
17:       $\text{timer}_i \leftarrow \text{millis}()$ 
18:       $\text{interval}_i \leftarrow \text{RANDOM}(5000, 10000)$ 
19:    end if
20:  end for
21: end loop

```

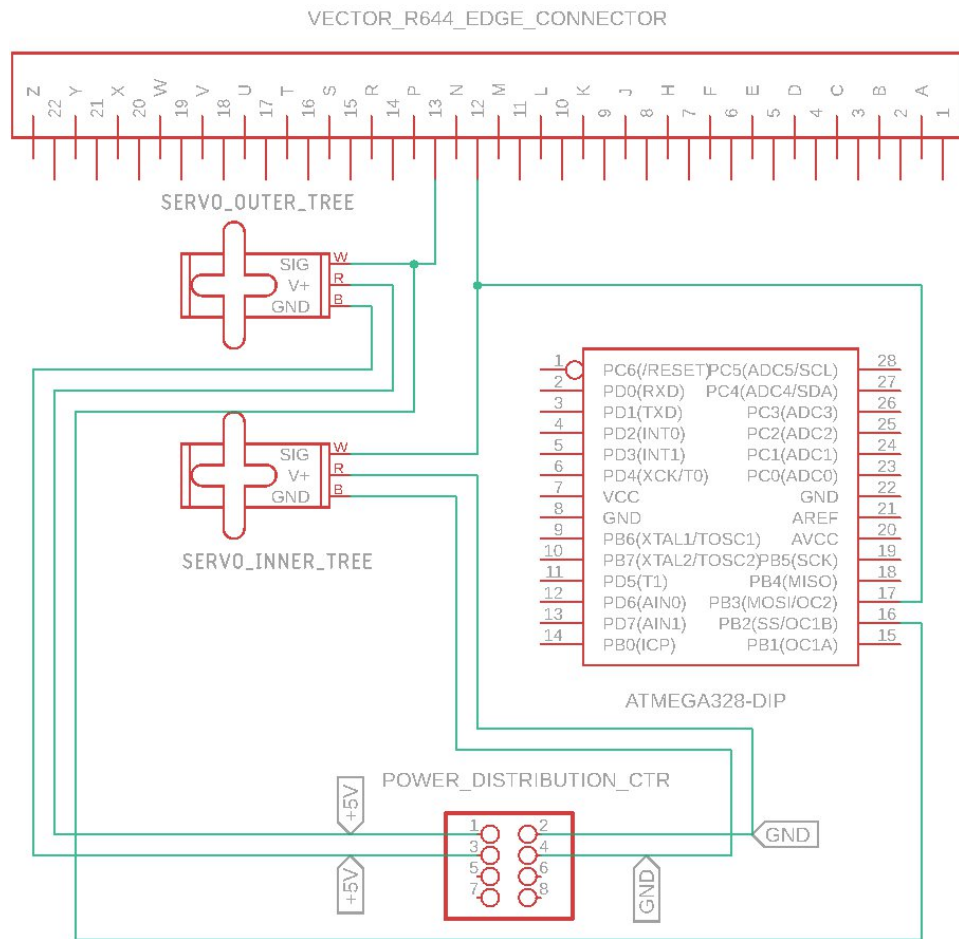


Figure 9: Servo Electrical Schematic

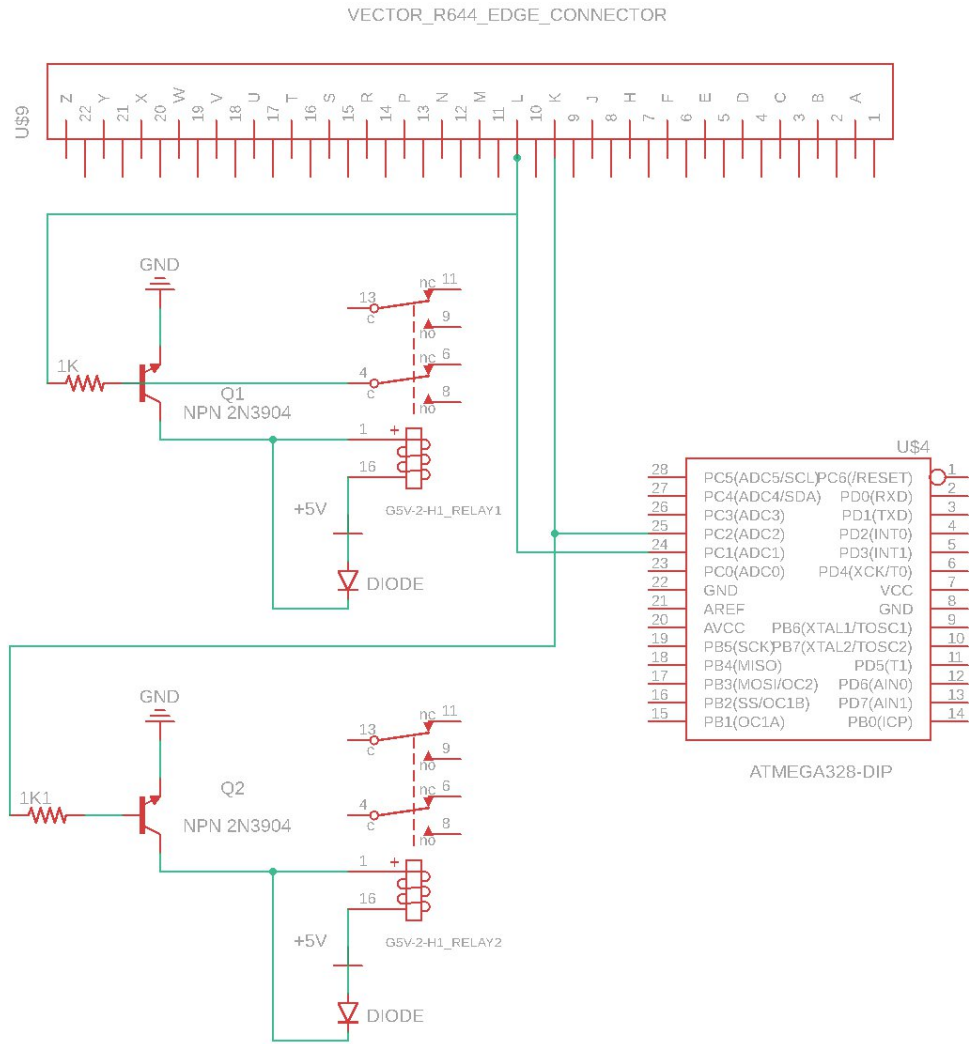


Figure 10: Two Power-On Relays

Trickle Charge Circuit The trickle charge circuit uses capacitor and a relay to throw a turnout (track switch) using low-power logic (TTL) control signals. This provides the main set of logic to control the outer or inner loop patterns of the train. As is described in the diagram, the capacitor's trigger to switch the train into a new pattern is a tradeoff between the amount of energy stored versus the time for the capacitor to charge again, motivating long delays between each pattern of the train's movement.

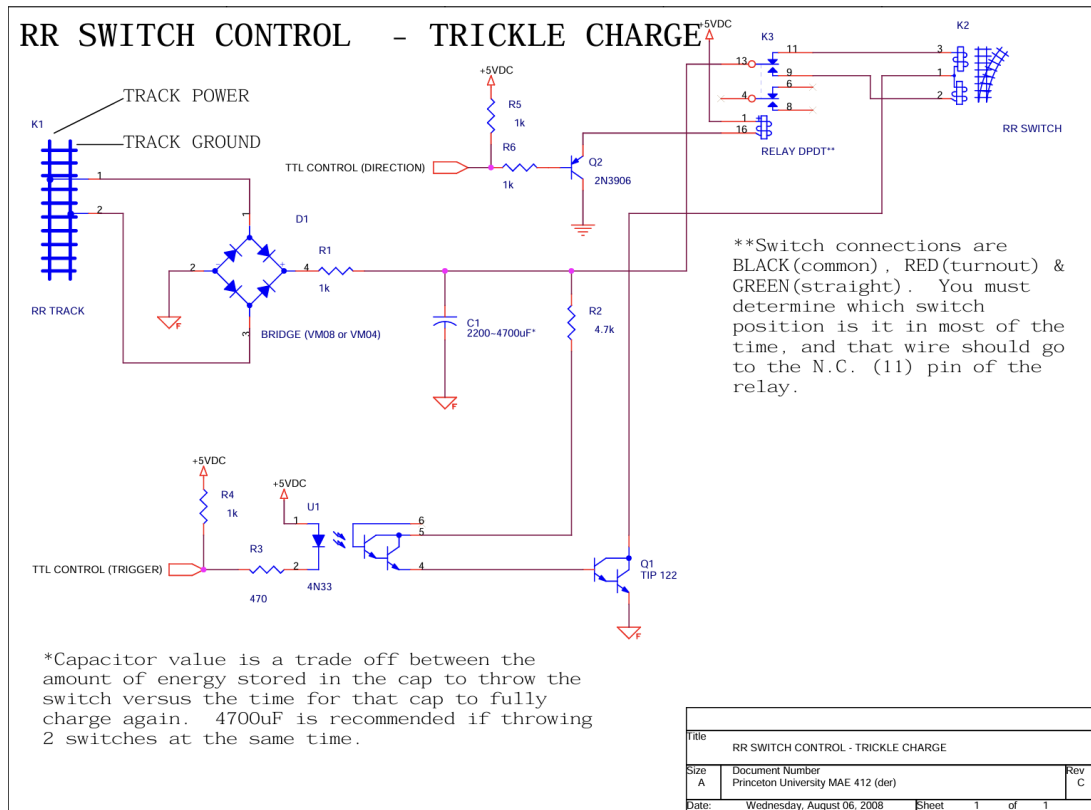


Figure 11: Trickle Charge Circuit for Switch Control

4.3 Implementing the Track Layout

The track layout was realized iteratively. While the AnyRail-designed layout provided a general model, a fully physical implementation using Atlas flex track, with iterative adjustment of curvature, provided a more customizable way to develop turnouts subject to real-world constraints. These tracks were installed incrementally using specific wood nails and a hammer provided by the lab staff. Each rail element was appended to the next with conductive rail joiner pieces, except for two track elements where the obstacles were, in which case these were joined to the system using insulated rail pieces so as to be able to control rail power incrementally.

4.4 Iterative Design of Obstacles and Mounts: 3D Printing

To assist with installing servos and obstacles on the track, we built CAD models of mounts using Fusion 360 and printed the models using the BambuLab printers in the lab. These mounts, for the servos, protoboards, and other components, as well as the tree obstacles themselves, were designed with respect to the shape of the governing system (e.g. the servo diameter for the screws into the obstacle for installation, or the thickness of the protoboard for the slit distance into the mount).

5 Results

Demo can be found [here](#). Further videos demonstrating the success of the system can be found [here](#). All code for this project is available in the Github repository linked [here](#).

In particular, we achieved several consecutive runthroughs on our project board (see videos 01 and 02 in the above folder), and on the test stand (see video 03). Lastly, we also were able to read the train barcode and clear it from the track (see video 04).

6 Conclusion

6.1 Concluding Notes

This project demonstrated an end-to-end autonomous railroad safety system on a scaled, two-loop heritage railroad layout, using a custom-built 6502-based vector-board computer alongside an Arduino co-processor to sense fallen-tree hazards, command turnout switching, and gate track power in real time. The primary objectives stated at the outset were met: the system reliably detects obstacles in two independent hazard zones via VL53L0X time-of-flight sensors, actuates obstacle deployment and removal through a servo-driven pulley mechanism with consistent positional repeatability, and executes a closed-loop response that halts or reroutes the train without human intervention. The fail-safe behavior, in which both track segments are de-energized when both hazard zones are simultaneously obstructed, was verified in repeated demonstration runs (see linked video).

Beyond the technical demonstration, the project exercised the iterative engineering-design process described in Section 2: the sensor modality, track layout, and obstacle-deployment mechanism each underwent meaningful revision between the initial proposal and the final realized system, and each revision was traceable to a specific constraint or design driver identified during bench testing. The architectural decision to delegate real-time peripheral I/O to an Arduino while reserving the 6502 for supervisory tasks proved practically important. Without it, integrating the time-of-flight sensors, servos, relays, and turnout drivers within the project timeline would have been considerably more difficult. The limitations of the system including the 500 ms control-loop period, the single-sensor-per-zone design, and the lack of higher-level path planning across multiple trains are well-defined and motivate the future work described in the next section.

6.2 Limitations, Next Steps & Future Work

One major limitation of the project came from the edge connector, which suffered a loose connection to the umbilical cord that caused our system to malfunction on demonstration day. We hypothesize that this is because our vector board mount put a strain on the vector board and edge connector, eventually breaking the soldered wire connections. This problem could have been avoided by mounting the vector board differently, or soldering wires to the edge connector more securely.

Secondly, the train route was determined by a fixed switch schedule, in which switches were set to specific configurations every 60 seconds. Unfortunately, this meant that sometimes the train was in the outer loop when the switches took the "inner loop" configuration, causing the train to derail at the switches.

Aspects of future work, as briefly alluded to earlier, include the following:

- **Train location-based switch logic** Instead of predetermining the switch schedule, we could set switches according to the train's position on the track. This could be accomplished by including hall effect sensors that could trigger track switches.
- **Tree-based switch logic** In the current project, we avoid collisions by stopping the train if it is in a hazard zone when a tree falls. Instead, we could also use the switches to reroute the train to a different loop to avoid the obstacle. This would also require hall effect sensors in the track.
- **Vision-Based Obstacle Detection.** Extend the current time-of-flight sensing system with an onboard camera and lightweight computer vision model to detect obstacles
- **Multi-Train Coordination and Scheduling.** Expand the system from a single-train controller to a multi-train environment, which would require dedicated collision avoidance software
- **Predictive Hazard Anticipation.** Incorporate temporal modeling of obstacle occurrence to transition from reactive control to predictive safety (perhaps with Control Barrier Functions, or CBFs) based on historical sensor data

References

- [1] Arduino. Arduino Uno Rev3. <https://docs.arduino.cc/hardware/uno-rev3/>, n.d. Product documentation. Accessed: 2026-04-23.
- [2] Atmel Corporation. *ATmega48A/PA/88A/PA/168A/PA/328/P: 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash (abridged)*. Abridged datasheet, distributed via the Princeton MAE 412 course handouts. Accessed: 2026-04-23.
- [3] Atmel Corporation. *AT28C64B: 64K (8K × 8) Parallel EEPROM with Page Write and Software Data Protection*, 2008. Datasheet. Accessed: 2026-04-23. Course-hosted copy unavailable; upstream manufacturer copy cited.
- [4] Integrated Device Technology, Inc. *IDT6116SA/LA: CMOS Static RAM 16K (2K × 8-bit)*. Datasheet, distributed via the Princeton MAE 412 course handouts. Accessed: 2026-04-23.
- [5] Lattice Semiconductor Corporation. *GAL16V8: High Performance E²CMOS PLD Generic Array Logic*. Datasheet, distributed via the Princeton MAE 412 course handouts. Accessed: 2026-04-23.
- [6] Andrew Lettenberger. Mt. Rainier Scenic Railroad returns to operation. <https://www.trains.com/trn/railroads/heritage-rail/heritage-railroads/mt-rainier-scenic-railroad-returns-to-operation/>, 2023. Accessed: 2026-04-21.
- [7] Rockwell International. *R6522 Versatile Interface Adapter (VIA)*. Datasheet, distributed via the Princeton MAE 412 course handouts. Accessed: 2026-04-23.
- [8] Rockwell International. *R6551 Asynchronous Communications Interface Adapter (ACIA)*. Datasheet, distributed via the Princeton MAE 412 course handouts. Accessed: 2026-04-23.
- [9] STMicroelectronics. *VL53L0X: World's Smallest Time-of-Flight Ranging and Gesture Detection Sensor*, 2018. Datasheet, document number DS11555 Rev. 5. Accessed: 2026-04-23.
- [10] Synertek, Inc. *SY6500: 8-Bit Microprocessor Family*. Datasheet, distributed via the Princeton MAE 412 course handouts. Accessed: 2026-04-23.
- [11] Texas Instruments. *74LS244: Octal Buffers and Line Drivers with 3-State Outputs*. Datasheet, distributed via the Princeton MAE 412 course handouts. Accessed: 2026-04-23.
- [12] Volker Weissflog and Frank Van Den Bossche. vasm portable and retargetable assembler. <http://sun.hasenbraten.de/vasm/>, n.d. Accessed: 2026-04-23.

A Sensing, Actuation, & Control Firmware Script

The following algorithm provides a complete walkthrough of the Arduino firmware, covering system initialization, the hold phase, sensor-driven track safety, servo-actuated obstacle deployment, turnout pattern sequencing, and telemetry output. All five subsystems execute within a single main loop iteration, gated by a 500 ms blocking delay at the loop's end.

Algorithm 3 Complete Arduino firmware - `setup()` and `loop()`

```
1: SETUP
2: Begin serial communication at 9600 baud; wait 3 s for monitor
3: Attach  $s_{11}$  (inner servo) to pin 11; attach  $s_{10}$  (outer servo) to pin 10
4:  $s_{11}.\text{WRITE}(90)$ ;  $s_{10}.\text{WRITE}(0)$  ▷ initial resting positions
   //  $I^2C$  sensor initialization
5: Drive  $\text{XSHUT}_{\text{outer}}$  (pin 3) and  $\text{XSHUT}_{\text{inner}}$  (pin 2) Low; wait 500 ms
6: Start  $I^2C$  bus at 400 kHz
7: Drive  $\text{XSHUT}_{\text{outer}}$  HIGH; wait 500 ms
8: Initialize  $S_{\text{out}}$ ; assign address 0x30; start continuous ranging
9: Drive  $\text{XSHUT}_{\text{inner}}$  HIGH; wait 500 ms
10: Initialize  $S_{\text{inn}}$ ; assign address 0x31; start continuous ranging
    // Track-power relay initialization
11: Set pins A1, A2 as outputs; write both LOW
    // Turnout switch initialization
12: for each switch  $i \in \{\text{SW1}, \text{SW2}, \text{SW3}, \text{SW4}\}$  do
13:     Set DIR pin and TRIG pin as outputs
14:     Write TRIG pin HIGH
15: end for
16: Record  $t_{\text{start}} \leftarrow \text{millis}()$ 

17: HELPER: MoveServoSlow( $s$ ,  $\text{startPos}$ ,  $\text{endPos}$ ,  $\tau$ )
18: if  $\text{startPos} < \text{endPos}$  then
19:     for  $\text{pos} \leftarrow \text{startPos}$  to  $\text{endPos}$  do
20:          $s.\text{WRITE}(\text{pos})$ ;  $\text{DELAY}(\tau)$ 
21:     end for
22: else
23:     for  $\text{pos} \leftarrow \text{startPos}$  downto  $\text{endPos}$  do
24:          $s.\text{WRITE}(\text{pos})$ ;  $\text{DELAY}(\tau)$ 
25:     end for
26: end if

27: HELPER: SetRelayStates( $\text{states}[4]$ )
28: for each switch  $i \in \{\text{SW1}, \text{SW2}, \text{SW3}, \text{SW4}\}$  do
29:     Write  $\text{DIR}_i \leftarrow \text{states}[i]$ 
30: end for

31: HELPER: TriggerAllStaggered()
32: for each switch  $i \in \{\text{SW1}, \text{SW2}, \text{SW3}, \text{SW4}\}$  do
33:     Write  $\text{TRIG}_i$  Low;  $\text{DELAY}(5 \text{ ms})$ ; Write  $\text{TRIG}_i$  HIGH;  $\text{DELAY}(5 \text{ ms})$ 
34: end for

35: HELPER: RunPattern( $\text{states}[]$ ,  $\text{delayTime}$ )
36: if not  $\text{patternTriggered}$  then
37:      $\text{SETRELAYSTATES}(\text{states})$ ;  $\text{TRIGGERALLSTAGGERED}$ 
38:      $\text{patternStartTime} \leftarrow \text{millis}()$ ;  $\text{currentDelay} \leftarrow \text{delayTime}$ ;  $\text{patternTriggered} \leftarrow \text{true}$ 
39: end if
40: if  $\text{millis}() - \text{patternStartTime} \geq \text{currentDelay}$  then
41:      $\text{currentPattern} \leftarrow \text{currentPattern} + 1$ ;  $\text{patternTriggered} \leftarrow \text{false}$ 
42: end if
```

Algorithm 4 Complete Arduino firmware - `setup()` and `loop()` (continued)

```
1: MAIN LOOP (repeats every  $\approx 500$  ms)  
  // Phase 1: Hold  
2: if not started and millis() - tstart < 5000 then  
3:   return  
4: end if  
5: if not started then  
6:   innerTimer  $\leftarrow$  millis(); innerInterval  $\leftarrow$  RANDOM(1000, 5000)  
7:   outerTimer  $\leftarrow$  millis(); outerInterval  $\leftarrow$  RANDOM(5000, 10000)  
8:   started  $\leftarrow$  true  
9: end if  
  // Phase 2: Sensor read  
10:  $d_{\text{out}} \leftarrow S_{\text{out}}.\text{READRANGECONTINUOUS}$ ;  $d_{\text{inn}} \leftarrow S_{\text{inn}}.\text{READRANGECONTINUOUS}$   
11: outerFallen  $\leftarrow (d_{\text{out}} \leq 170)$ ; innerFallen  $\leftarrow (d_{\text{inn}} \leq 120)$   
  // Phase 3: Track safety  
12: Write A1  $\leftarrow$  (innerFallen ? HIGH : LOW)  
13: Write A2  $\leftarrow$  (outerFallen ? HIGH : LOW)  
14: if innerFallen and outerFallen then  
15:   SIGNAL(BOTHTRACKSCUT)  
16: else if innerFallen then  
17:   SIGNAL(TRACK4CUT)  
18: else if outerFallen then  
19:   SIGNAL(TRACK5CUT)  
20: end if  
  // Phase 4: Servo actuation  
21: if millis() - innerTimer  $\geq$  innerInterval then  
22:   innerState  $\leftarrow \neg$  innerState  
23:   MOVESERVOSLOW( $s_{11}$ , (innerState = 1 ? 0 : 110), (innerState = 1 ? 110 : 0), 15)  
24:   innerTimer  $\leftarrow$  millis(); innerInterval  $\leftarrow$  RANDOM(5000, 10000)  
25: end if  
26: if millis() - outerTimer  $\geq$  outerInterval then  
27:   outerState  $\leftarrow \neg$  outerState  
28:   MOVESERVOSLOW( $s_{10}$ , (outerState = 1 ? 110 : 0), (outerState = 1 ? 0 : 110), 15)  
29:   outerTimer  $\leftarrow$  millis(); outerInterval  $\leftarrow$  RANDOM(5000, 10000)  
30: end if  
  // Phase 5: Turnout pattern sequencer  
31: if currentPattern = 0 then  
32:   RUNPATTERN({L,L,L,H}, 60 000 ms) ▷ Pattern 1: outer loop, 60 s  
33: else if currentPattern = 1 then  
34:   RUNPATTERN({H,H,H,H}, 30 000 ms) ▷ Pattern 2: inner loop, 30 s  
35: else if currentPattern = 2 then  
36:   RUNPATTERN({H,L,H,L}, 30 000 ms) ▷ Pattern 3: default loop, 30 s  
37: else  
38:   currentPattern  $\leftarrow$  0  
39: end if  
  // Phase 6: Telemetry  
40: Print currentPattern,  $d_{\text{inn}}$ ,  $d_{\text{out}}$ , innerFallen, outerFallen over serial  
  // Phase 7: Loop delay  
41: DELAY(500 ms)
```

B Individual Contributions

Project:

Section	Member(s)
Vector board computer	Hriday Unadkat, Sonal Bhatia, Sanjana Venkatesh
Track layout	Hriday Unadkat & Sonal Bhatia
Auxiliary circuitry	Sanjana Venkatesh
Software	Sonal Bhatia & Hriday Unadkat
Tree and mount design and fabrication	Hriday Unadkat
Board wiring	Sanjana Venkatesh, Hriday Unadkat, Sonal Bhatia

Report writing:

Section	Member(s)
Abstract	Sonal Bhatia & Sanjana Venkatesh & Hriday Unadkat
Introduction & Theory	Sonal Bhatia & Hriday Unadkat
Engineering Design	Sonal Bhatia & Hriday Unadkat
Concept of Operations	Sonal Bhatia & Hriday Unadkat
Methodology	Sonal Bhatia & Hriday Unadkat
Computer Design	Sonal Bhatia & Sanjana Venkatesh & Hriday Unadkat
Track Design	Sonal Bhatia & Hriday Unadkat
Modifications	Sonal Bhatia & Sanjana Venkatesh
Conclusion	Hriday Unadkat & Sonal Bhatia & Sanjana Venkatesh

Table 2: Team members' contributions to report writing.